TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN
XÂY DỰNG CHATBOT HỎI ĐÁP SÁCH

Môn: Nhập môn xử lý ngôn ngữ tự nhiên

Lớp 22CNTThuc

***Giáo viên hướng dẫn*:**

Nguyễn Hồng Bửu Long

Lê An Vinh

Đinh Điền

Thành phố Hồ Chí Minh, tháng 01 năm 2025

THÀNH VIÊN NHÓM

TRẦN PHÚC HẢI - 22127096

LÊ NGUYỄN GIA BẢO - 22127029

ĐINH NGUYỄN QUỲNH HƯƠNG - 22127146

MỤC LỤC

MỞ ĐẦU 5

QUY TRÌNH TẠO TẬP DỮ LIỆU HỎI-ĐÁP (QA) 6

I. Giai đoạn 1 – Phân bổ chương và nội dung 6

1. Trích xuất thông tin cơ bản của sách 7

2. Trích xuất chương sách từ mục lục 7

3. Xây dựng cấu trúc phân cấp chương 7

4. Phân bổ nội dung theo chương sách 9

II. Giai đoạn 2 – Xử lý ngữ cảnh 12

2. Chia nhỏ dữ liệu – Data Chunking 12

3. Tóm tắt dữ liệu 13

III. Giai đoạn 3 – Tạo cặp Hỏi-Đáp (QA) 13

1. Tạo câu hỏi và câu trả lời: 14

2. Chuẩn hóa định dạng: 14

TRUY XUẤT VÀ TẠO CÂU TRẢ LỜI DỰA TRÊN TẬP DỮ LIỆU QA SẴN CÓ 15

I. Tạo vector biểu diễn ngữ nghĩa (embedding) 15

II. Xây dựng chỉ mục tìm kiếm ngữ nghĩa 15

III. Truy xuất ngữ cảnh liên quan và tạo câu trả lời 15

ĐÁNH GIÁ HIỆU QUẢ CỦA MÔ HÌNH SỬ DỤNG BỘ TIÊU CHÍ RAGAS 17

I. Giới thiệu 17

1. Context Recall – Độ phủ của ngữ cảnh 17

2. Factual Correctness – Tính chính xác về mặt dữ kiện 18

3. Faithfullness – Tính trung thực/Độ tin cậy 18

4. Semantic Similarity – Độ tương đồng ngữ nghĩa 18

II. Cài đặt 19

1. Dữ liệu đầu vào 19

2. Sinh tập phản hồi 20

3. Đánh giá mô hình 22

III. Kết quả 23
1. Tổng quan 23
2. Chi tiết 23
3. Nhận xét 24
XÂY DỰNG ỨNG DỤNG WEB CHATBOT VỚI GIAO DIỆN NGƯỜI DÙNG 26
I. Thiết kế giao diện 26
II. Xử lý logic 27
TÀI LIỆU THAM KHẢO 28

MỞ ĐẦU

Trong thời đại công nghệ 4.0, trí tuệ nhân tạo (AI) ngày càng khẳng định vai trò quan trọng trong việc cải thiện hiệu quả và nâng cao trải nghiệm người dùng trên nhiều lĩnh vực. Một trong những ứng dụng nổi bật của AI là ChatBot, công cụ hỗ trợ giao tiếp tự động, được tích hợp rộng rãi trong dịch vụ khách hàng, giáo dục, thương mại điện tử, và nhiều lĩnh vực khác. Trong bối cảnh đó, việc xây dựng ChatBot có khả năng cung cấp thông tin nhanh chóng và chính xác đã trở thành một nhu cầu thiết yếu.

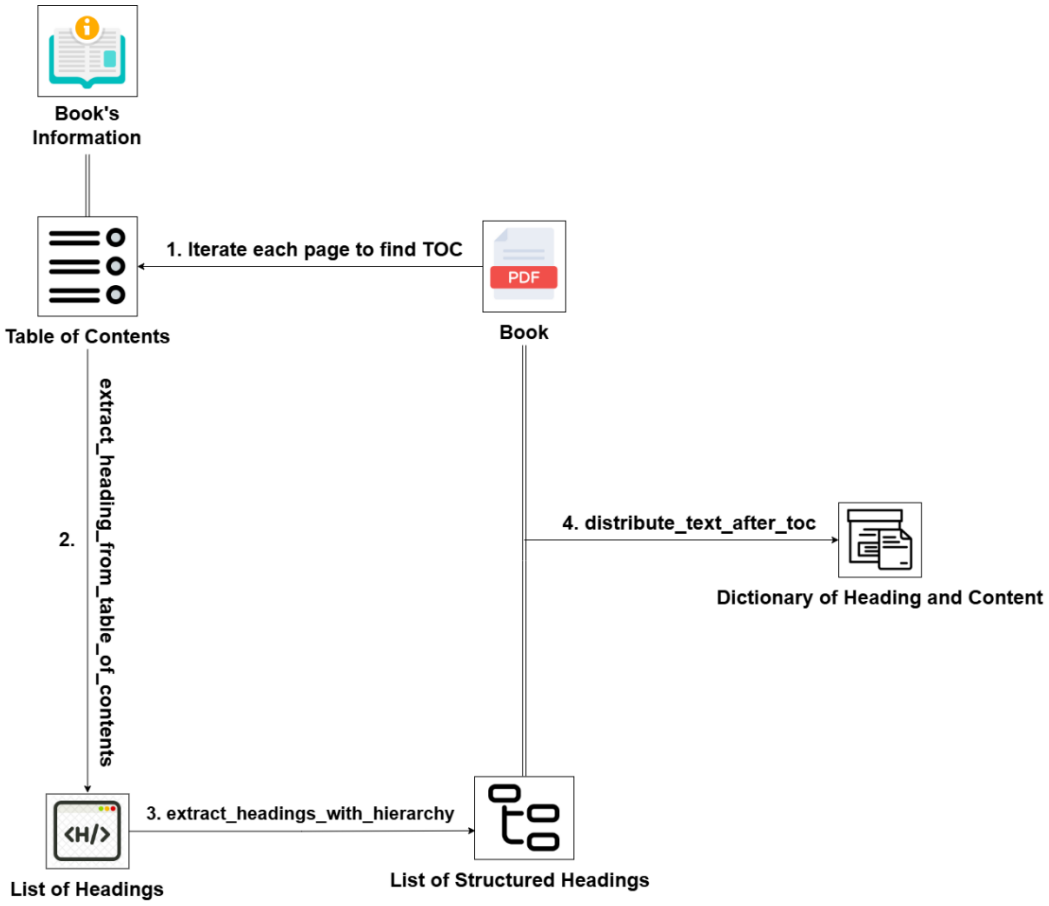
Đồ án "Xây dựng ChatBot hỏi đáp sách" tập trung vào việc thiết kế và triển khai một hệ thống ChatBot hỗ trợ tìm kiếm và trả lời câu hỏi dựa trên nội dung sách. Với mục tiêu này, nhóm thực hiện đã nghiên cứu và áp dụng các phương pháp trong xử lý ngôn ngữ tự nhiên (NLP) vào việc trích xuất thông tin, cấu trúc hoá nội dung, tạo các cặp câu hỏi – câu trả lời (QA), đánh giá kết quả tạo sinh, xây dựng chatbot để dựa vào QA database trả lời user queries, và xây dựng website cho ứng dụng.

Báo cáo này sẽ trình bày chi tiết các giai đoạn phát triển đề tài, từ xây dựng tập dữ liệu, xây dựng ứng dụng, đến đánh giá hiệu quả, qua đó đề xuất một giải pháp thực tiễn và hiệu quả cho việc tối ưu hóa việc truy xuất tri thức từ tài liệu sách.

QUY TRÌNH TẠO TẬP DỮ LIỆU HỎI-ĐÁP (QA)

I. Giai đoạn 1 – Phân bổ chương và nội dung

- **Mục tiêu:** Giai đoạn này nhằm mục đích trích xuất cấu trúc logic và nội dung của tài liệu PDF, chuyển đổi thành định dạng JSON để thuận tiện cho các bước xử lý tiếp theo.
- **Đầu vào:** Sách dạng PDF.
- **Đầu ra:** Tệp JSON chứa thông tin về cấu trúc chương, thông tin cơ bản của sách (tên sách, tác giả, năm xuất bản) và nội dung text của từng chương sách.



Hình 1: Quy trình trích xuất thông tin từ một cuốn sách PDF

1. **Trích xuất thông tin cơ bản của sách**

- Bước đầu tiên là trích xuất metadata quan trọng từ tài liệu PDF. Quá trình này được thực hiện theo hai phương pháp:
  - **Sử dụng thư viện PyMuPDF:** PyMuPDF cho phép **đọc và thao tác với file PDF như một cấu trúc dữ liệu có tổ chức**, thay vì chỉ là một chuỗi văn bản đơn thuần. Nhờ vậy, PyMuPDF có thể **trích xuất chính xác** không chỉ văn bản mà còn cả các thông tin khác (metadata) như **tiêu đề sách, tác giả, năm xuất bản, mục lục (với các chương và số trang tương ứng)**, v.v.
  - **Sử dụng công nghệ OCR:** Đối với các tài liệu PDF dạng ảnh (scanned documents) hoặc vừa ảnh vừa text, Google Cloud Vision API được sử dụng để chuyển đổi hình ảnh thành văn bản.
- Metadata quan trọng bao gồm:
  - **title:** Tên sách.
  - **author:** Tên tác giả.
  - **year:** Năm xuất bản của sách.
- **Tối ưu hóa và hiệu chỉnh:** Văn bản được trích xuất từ PyMuPDF hoặc OCR sẽ được đưa vào API của GPT để sửa lỗi ngữ pháp và chính tả, đảm bảo độ chính xác và nhất quán của văn bản.

2. **Trích xuất chương sách từ mục lục**

- Mục lục cung cấp thông tin quan trọng về cấu trúc logic của tài liệu. Do đó, bước tiếp theo là trích xuất các chương sách từ mục lục. Quá trình này được thực hiện bằng hàm **extract_heading_from_table_of_contents(pdf_path)**. Kết quả trả về là một biến kiểu dữ liệu dictionary **headings_info** chứa thông tin chi tiết về mỗi chương sách.
- **Thông tin chương sách lúc này sẽ bao gồm: heading** – tên chương, **page_num** – số trang tương ứng với chương, và ba thông tin siêu dữ liệu đã tìm được trước đó là **title** – tên sách, **author** – tên tác giả, **year** – năm xuất bản của sách.

3. **Xây dựng cấu trúc phân cấp chương**

- Để hiểu rõ mối quan hệ giữa các chương, danh sách chương phẳng được chuyển đổi thành cấu trúc phân cấp. Hàm **extract_headings_with_hierarchy(headings_info)** thực hiện nhiệm vụ này, tạo ra biến kiểu dữ liệu dictionary **structured_headings** biểu diễn cấu trúc phân cấp.

- **Bước 1 – Chuẩn bị:** Ta có một danh sách các chương chưa được sắp xếp, một danh sách **structured_headings** để đặt các chương đã được sắp xếp, một stack chứa các chương sách đang chờ được xếp đúng cấp độ.
- **Bước 2 – Sắp xếp từng chương sách:**
  - **Lấy từng chương sách ra khỏi danh sách chưa được sắp xếp.**
  - **Xác định cấp độ của chương sách:**
    - Nếu chương sách có đánh số (ví dụ: 1.; 1.1; 2.; a), cấp độ của chương sách sẽ dựa vào số lượng dấu chấm trong phần đánh số (nhiều dấu chấm hơn nghĩa là hộp nhỏ hơn). Ví dụ: "1.2.3." sẽ nhỏ hơn "1.2" và "1."
    - Nếu chương sách không được đánh số, chúng ta sẽ tạm thời coi là nó có cấp độ lớn nhất (cấp độ 1).
    - Hạn chế của phương pháp trên:
      Giả định về cách đánh số nhất quán: Phương pháp này dựa trên giả định rằng cách đánh số tiêu đề là nhất quán và logic trong toàn bộ tài liệu. Nếu cách đánh số thay đổi hoặc không tuân theo một quy tắc rõ ràng, việc xác định cấp độ sẽ không chính xác. Ví dụ:
      Sử dụng số La Mã (I, II, III) cho cấp 1, rồi lại dùng số Ả Rập (1, 2, 3) cho cấp 2.
      Đánh số không liên tục, nhảy cóc.

      Không phân biệt được các trường hợp đặc biệt: Tiêu đề không được đánh số nhưng có mối quan hệ phân cấp. Có những trường hợp tiêu đề không được đánh số nhưng vẫn có mối quan hệ cha-con. Ví dụ:

      Phần I: Tổng quan

      Giới thiệu

      Mục tiêu nghiên cứu

      Phần II: Phân tích

      Phương pháp

      Kết quả
  - **Tìm chương sách cấp lớn hơn:**
    - Nhìn vào stack, nếu trong stack có chương sách nào lớn hơn chương sách hiện tại thì chương đó sẽ là chương sách cấp cha.

        - Nếu trong stack có nhiều chương sách có cấp độ lớn hơn, thì chương sách đầu tiên trong stack sẽ là chương sách cấp cha.
        - Nếu trong stack không có chương sách nào, thì chương sách hiện tại đang xét sẽ có cấp độ lớn nhất.
- Thông tin trong cấu trúc phân cấp bao gồm:
    - **heading:** Tên chương.
    - **heading_level:** Vị trí của chương sách trong cấu trúc phân cấp của tài liệu. Nó được biểu diễn bằng một số nguyên. Số càng nhỏ, cấp bậc càng cao (gần gốc hơn). Cấp 1 thường là các phần chính (Part) hoặc chương lớn (Chapter). Cấp 2 là các chương nhỏ hơn hoặc các mục lớn bên trong chương. Cấp 3 là các mục nhỏ hơn bên trong cấp 2, và cứ tiếp tục như vậy.
    - **larger_headings:** Danh sách các chương sách cấp cha của chương hiện tại (có **heading_level** nhỏ hơn).
    - **page_num:** Số trang tương ứng.
    - **Các thông tin siêu dữ liệu đã tìm được ở các bước trên.**

4. Phân bổ nội dung theo chương sách

- Bước cuối cùng của giai đoạn này là liên kết nội dung văn bản với các chương sách tương ứng. Dựa vào cấu trúc phân cấp số trang, nội dung được gán cho các chương sách phù hợp, tạo mối quan hệ cha-con giữa chương sách và nội dung.
- **Mục tiêu:** Gán nội dung văn bản từ tài liệu PDF cho các chương đã được trích xuất và cấu trúc ở các bước trước.
- **Phương pháp:**
    - **Xác định ranh giới nội dung:** Dựa vào số trang của các chương trong cấu trúc phân cấp (**structured_headings**) và trang cuối cùng của mục lục (**toc_end_page**), xác định ranh giới nội dung cho từng chương. Nội dung của một chương bắt đầu từ trang chứa tên chương đó (hoặc trang ngay sau trang cuối cùng của mục lục) và kết thúc ở trang ngay trước trang chứa tên chương tiếp theo (hoặc cuối tài liệu).
        - **Ví dụ:** Giả sử cuốn sách có cấu trúc như sau:
            - **Trang 5:** Chương 1: Giới thiệu
            - **Trang 10:** Chương 2: Phương pháp nghiên cứu
            - **Trang 20:** Chương 3: Kết quả
            - **Trang 30:** (hết tài liệu)
            - Trong trường hợp này:

Nội dung **Chương 1** bắt đầu từ **trang 5** (trang chứa tiêu đề "Chương 1: Giới thiệu") và kết thúc ở **trang 9** (trang ngay trước trang chứa tiêu đề "Chương 2: Phương pháp nghiên cứu").
Nội dung **Chương 2** bắt đầu từ **trang 10** và kết thúc ở **trang 19**.
Nội dung **Chương 3** bắt đầu từ **trang 20** và kết thúc ở **trang 30** (cuối tài liệu).

- Sử dụng hàm **distribute_text_after_toc** để bắt đầu phân bổ nội dung vào chương tương ứng:
  - **Duyệt qua từng trang:** Lặp qua từng trang PDF sau trang cuối cùng của mục lục.
  - **Loại bỏ header và footer:** Loại bỏ phần header và footer dựa vào ngưỡng **header_margin** và **footer_margin** đã được xác định. Việc loại bỏ này tránh nhầm lẫn nội dung header/footer với nội dung chính. Header và footer thường chứa heading (đã được lưu trước đó, nếu cần thì truy xuất), nên loại bỏ khỏi nội dung sách để tránh gây nhiễu, dài dòng.
    - Ví dụ minh họa: Giả sử trang PDF có kích thước 612 x 792 (rộng x cao), header_margin là 70 và footer_margin là 50.
      page_bbox sẽ là (0, 0, 612, 792).
      crop_box sẽ là (0, 0 + 70, 612, 792 - 50) tương đương với (0, 70, 612, 742).
      Như vậy, phần trang có tọa độ y từ 0 đến 70 (header) và từ 742 đến 792 (footer) sẽ bị loại bỏ. Phần còn lại (từ y = 70 đến y = 742) chính là nội dung chính của trang và sẽ được trích xuất văn bản.
  - **Xử lý trang dạng ảnh**: Nếu trang không có text (trường hợp trang chỉ có ảnh), thực hiện OCR bằng hàm **perform_ocr_on_page()** để chuyển đổi thành văn bản. Cũng sử dụng Google Cloud Vision API tương tự ở trên.
  - **Phân bổ nội dung dựa trên sự xuất hiện của chương**: Duyệt qua từng dòng văn bản đã được làm sạch, nếu phát hiện dòng đó khớp với một chương sách trong **structured_headings** (sử dụng hàm **clean_and_match_heading**), cập nhật **current_heading** bằng chương mới. Những dòng tiếp theo cho đến khi gặp chương mới khác sẽ được coi là nội dung thuộc về **current_heading** và được thêm vào phần nội dung tương ứng của chương đó trong biến kiểu dữ liệu dictionary **section_contents**.
  - **Xử lý các chương sách lồng nhau:** Cấu trúc phân cấp chương (**structured_headings**) giúp xác định mối quan hệ cha-con giữa các

chương. Khi phân bổ nội dung, ưu tiên gán nội dung cho chương con gần nhất.

- **Làm sạch nội dung text đã trích xuất:** Trước khi gán nội dung cho chương, thực hiện tiền xử lý để loại bỏ các ký tự đặc biệt, số thứ tự đầu dòng.

- **Kết quả:** Kết quả của bước này là danh sách **result** kiểu dữ liệu dictionary, mỗi dictionary đại diện cho một chương và thông tin liên quan, bao gồm:
  - **heading:** Tên chương
  - **heading_level:** Cấp độ của chương.
  - **larger_heading:** Danh sách các chương cấp cao hơn của chương hiện tại.
  - **content:** Phần nội dung văn bản đã được phân bổ cho chương này.
  - **page_num:** Số trang chứa chương sách tương ứng.
  - **title:** Tên sách.
  - **author:** Tên tác giả.
  - **year:** Thời gian xuất bản.


```
"Heading": "Đôi Lời Cùng Bạn Trẻ!",
"Cấp độ heading": 1,
"Các heading lớn hơn": [],
"Nội dung": "“Dữ quân nhất dạ thoại, thắng độc thập niên thư”. Đọc sách mười năm
không bằng một đêm được... “Chat” (tưởng tượng!) với các... cụ. Thật thế chăng, thưa
không dám chắc! Và làm sao có chuyện ấy được? Nhưng, vui và gây cảm hứng để các bạn
tự tiếp tục lên đường, đó là mục đích của... “chương trình phát sóng” này. Mong Quý
bạn vui vẻ đón nhận và chia sẻ! Bùi Văn Nam Sơn Danh mục tác phẩm của Descartes được
trích dẫn Trong sách này, tác phẩm của Descartes được trích dẫn thống nhất từ ŒUVRES
DE DESCARTES/ TÁC PHẨM CỦA DESCARTES (11 tập, bản mới, được ấn hành ở Paris từ 1981)
do Charles Adam và Paul Tannery ấn hành (nên thường được viết tắt là “AT”), kèm số
tập và số trang, theo thói quen của giới nghiên cứu về Descartes. Tên các tác phẩm
chính thường trích dẫn sẽ được viết tắt sau lần giới thiệu đầu tiên: Discours:
Discours de la Méthode/ Luận văn về Phương pháp (AT VI) Meditations: Meditationes de
prima philosophiae (La-tinh)/ Những suy niệm về Đệ nhất triết học/ Méditations
métaphysiques (bản dịch tiếng Pháp)/ Những suy niệm siêu hình học (AT VII) Principia:
Principia philosophiae (La-tinh)/ Les Principes de la philosophie (bản dịch tiếng
Pháp)/ Những Nguyên lý triết học (AT VIII - 1) Passions: Les Passions de l’âme/Những
xúc cảm của linh hồn (AT XI) tương ứng trên mạng. Được khuyến khích đọc là bộ The
Philosophical Writings of Descartes (do J. Cottingham ấn hành), Cambridge 1984-1991.
Trong tiếng Việt, có các bản dịch:“Phương pháp luận Descartes”(1973) và “Những suy
niệm siêu hình học của Descartes” (1962) của Giáo sư Trần Thái Đỉnh. 1",
"Số trang": "9",
"Tên sách": "Mmc..",
"Tác giả": "Bùi Văn Nam Sơn",
"Thời gian xuất bản": "1596-1650"
```


Hình 2: Ví dụ một record trong tệp đầu ra dạng JSON chứa thông tin về cấu trúc chương kèm metadata của sách

II. Giai đoạn 2 – Xử lý ngữ cảnh

- **Mục tiêu:** Chuẩn bị dữ liệu văn bản đã được phân bổ ở bước trước thành định dạng phù hợp cho việc tạo cặp QA, bao gồm việc chia nhỏ dữ liệu, tóm tắt và định dạng.
- **Đầu vào:** Tệp JSON từ giai đon 1 – Phân bổ chương và nội dung.
- **Đầu ra:** Tệp JSON chứa các context chunk đã được xử lý, sẵn sàng cho việc tạo cặp QA.

1. **Cấu hình API OpenAI:** Bước này bao gồm việc thiết lập và xác thực API key để sử dụng các dịch vụ của OpenAI, cụ thể là mô hình ngôn ngữ lớn như gpt-4o hay gpt-4o-mini.

2. **Chia nhỏ dữ liệu – Data Chunking**

- Nội dung văn bản (đã được phân bổ cho các chương ở giai đoạn 1) được chia thành các khối (chunks) nhỏ hơn, có độ dài tối đa 512 tokens, phù hợp với độ dài trung bình của context trong dataset ViQuAD, cũng như tránh trường hợp text quá dài, GPT không xử lý được. Quá trình này được thực hiện bởi hàm **chunk_data()**.
- Chi tiết các bước trong hàm:
  - **Duyệt qua dữ liệu đầu vào và trích xuất thông tin:**
    - Lấy chương sách (heading) và nội dung (content) từ mỗi phần tử.
    - Tách riêng phần metadata (các thông tin khác ngoài heading và content).
  - **Tiền xử lý:** Tính số lượng token (**original_tokens**) và số lượng ký tự (**original_chars**) của nội dung gốc để đảm bảo nội dung của ngữ cảnh không vượt quá giới hạn này.
  - **Kết hợp chương sách và nội dung và chia thành các đoạn văn và các câu.**
    - Nếu đoạn văn có số lượng token nhỏ hơn **max_output_tokens**, thêm trực tiếp đoạn văn đó vào danh sách **chunks**.
    - Nếu đoạn văn có số lượng token lớn hơn **max_output_tokens**, sử dụng hàm **split_into_sentences()** để chia đoạn văn thành các câu, đảm bảo tính toàn vẹn về ngữ nghĩa, chia nhỏ văn bản theo câu giúp đảm bảo rằng các khối được tạo ra không bị cắt giữa chừng một câu. Sau đó duyệt qua từng câu và ghép các câu lại thành các khối có độ dài tối đa **max_output_tokens** tokens.

- **Tạo context chunk:** Đối với mỗi khối (chunk) đã tạo, tạo một dictionary mới với các thông tin:
  - **Metadata.**
  - **context_type:** Nội dung văn bản đã được chia nhỏ.
  - **summary_level:** Cấp độ tóm tắt dữ liệu.
  - **summary_depth:** Độ sâu của quá trình tóm tắt dữ liệu.
  - **original_tokens:** Số lượng token ban đầu.
  - **original_chars:** Số lượng ký tự ban đầu.
  - **heading:** Tên chương.
  - **content:** Nội dung ứng với chương đó sau khi được tóm tắt.

3. Tóm tắt dữ liệu

- Để tạo ra các ngữ cảnh súc tích hơn, các context chunk được tóm tắt thông qua API của OpenAI. Quá trình này được thực hiện bởi hàm **summarize_data()**, với các tham số:
  - **data:** Dữ liệu đã được chia nhỏ ở bước 2.
  - **summary_level:** Chỉ định cấp độ tóm tắt văn bản.
  - **max_output_tokens:** Giới hạn số lương token tối đa cho mỗi lần tóm tắt (mặc định là 512 tokens).
  - **overlap_rate:** Tỷ lệ phần trăm trùng lặp giữa các lần tóm tắt liên tiếp (mặc định là 0.2).
- Chi tiết các bước trong hàm **summarize_data()**:
  - **Duyệt qua dữ liệu đầu vào và trích xuất thông tin:** Lấy nội dung content từ mỗi phần tử và tách riêng các thông tin metadata khác.
  - **Tiền xử lý và tính toán thông tin:** Tính toán số lượng token (**original_tokens**) và số lượng ký tự (**original_chars**) của nội dung gốc.
  - Tóm tắt nội dung bằng hàm **summarize_content()**. Chia nội dung thành các phần (chunk) có độ dài **chunk_chars**, với phần trùng lặp giữa các phần là **overlap_chars**. Với mỗi phần, gọi API của OpenAI để tóm tắt.

III. Giai đoạn 3 – Tạo cặp Hỏi-Đáp (QA)

- Mục tiêu: Tạo các cặp câu hỏi và câu trả lời dựa trên các context chunk đã được xử lý ở giai đoạn 2.
- Đầu vào: Tệp JSON chứa dữ liệu đã được xử lý ở giai đoạn 2.

- Đầu ra: Tệp Excel chứa danh sách các cặp Hỏi-Đáp.

1. **Tạo câu hỏi và câu trả lời:**

   Tính toán số lượng cặp QA, dựa vào số lượng token của ngữ cảnh và ước lượng số lượng token trung bình cho mỗi cặp QA, tính toán số lượng cặp QA cần tạo.

2. **Chuẩn hóa định dạng:**

- **question:** Câu hỏi được tạo ra từ ngữ cảnh.
- **answers:**
  - **text:** Câu trả lời, được trích xuất trực tiếp từ ngữ cảnh.
  - **answer_start:** Vị trí bắt đầu của câu trả lời trong ngữ cảnh gốc. Ví dụ context là "Hôm nay trời mưa" với câu hỏi là "Hôm nay trời mưa không?" thì answer là "trời mưa" với answer_start là 2 (Chữ đầu trong answer "trời" có index 2 trong context).

TRUY XUẤT VÀ TẠO CÂU TRẢ LỜI DỰA TRÊN TẬP DỮ LIỆU QA SẴN CÓ

- **Mục tiêu:** Xây dựng hệ thống ChatBot có khả năng truy xuất thông tin từ tập dữ liệu QA đã cho.

I. Tạo vector biểu diễn ngữ nghĩa (embedding)

- Sử dụng mô hình **Sentence-BERT (distiluse-base-multilingual-cased-v1)** để tạo ra các vector embedding cho phần **ngữ cảnh (context)** và **câu hỏi (question)** trong tập dữ liệu QA.
- Mô hình Sentence-BERT được lựa chọn vì khả năng tạo ra embedding chất lượng cao cho các câu, thể hiện được ngữ nghĩa và mối quan hệ giữa các câu.

II. Xây dựng chỉ mục tìm kiếm ngữ nghĩa

- Sử dụng thư viện **FAISS** để tạo index cho các vector embedding của **câu hỏi**.
- Chỉ mục FAISS (IndexFlatL2) cho phép tìm kiếm các vector gần nhất (theo khoảng cách Euclidean) với vector truy vấn một cách nhanh chóng và hiệu quả.
- Chỉ sử dụng embedding của câu hỏi để tạo index, vì ở bước này, chúng ta đã lưu trữ embedding của context và question riêng biệt.

III. Truy xuất ngữ cảnh liên quan và tạo câu trả lời

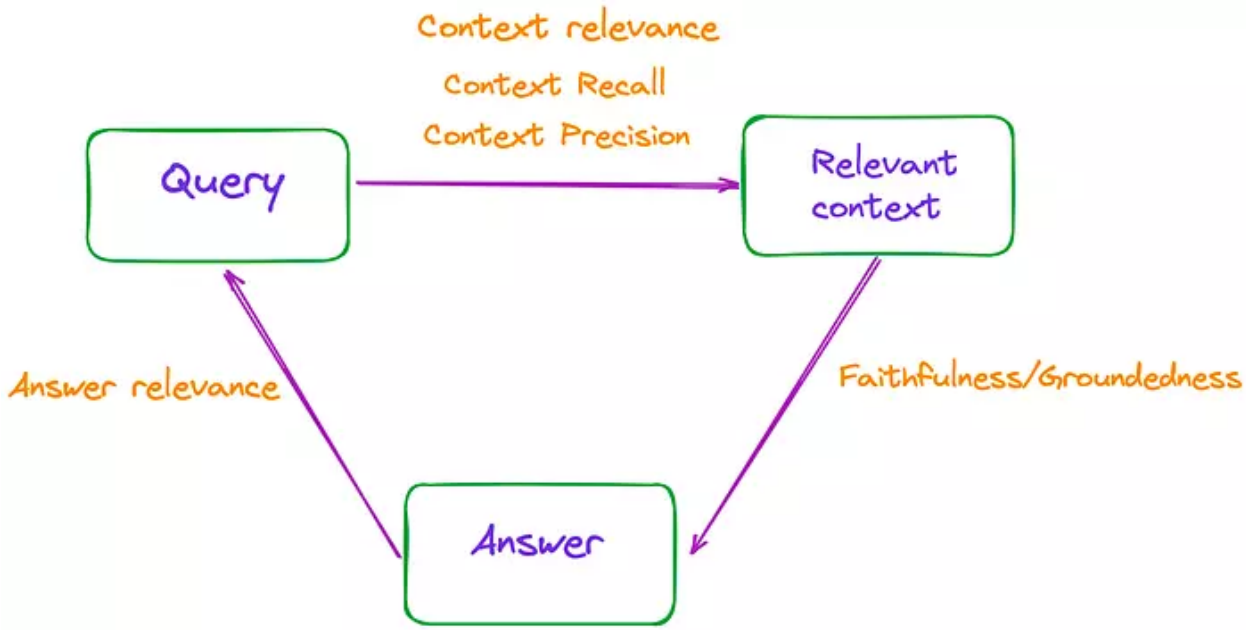
- Khi người dùng đặt câu hỏi, sử dụng mô hình Sentence-BERT để tạo ra vector embedding cho câu hỏi đó.
- Sử dụng chỉ mục FAISS để tìm kiếm k vector embedding của câu hỏi gần nhất với vector embedding của câu hỏi người dùng (có ngữ nghĩa gần nhất). Giá trị k mặc định là 5, nếu có thời gian cần optimize để chọn ra giá trị k tối ưu nhất.
- Dựa vào chỉ số (index) của các vector gần nhất, truy xuất các dòng tương ứng trong tập dữ liệu để lấy ra k ngữ cảnh liên quan nhất.
- Kết hợp k ngữ cảnh này thành một chuỗi context.
- Sử dụng API OpenAI (gpt-4o-mini) để tạo câu trả lời dựa trên:

- Lịch sử hội thoại: Bao gồm các tin nhắn trước đó giữa người dùng và hệ thống. Website có lưu lại conversation history và feed vào API cùng với query mới để đảm bảo flow của cuộc trò chuyện, và có đưa ra câu trả lời liên quan, không bị lạc đề.
- Câu hỏi hiện tại của người dùng.
- Các ngữ cảnh liên quan được lấy từ bước 3.

ĐÁNH GIÁ HIỆU QUẢ CỦA MÔ HÌNH SỬ DỤNG BỘ TIÊU CHÍ RAGAS

I. Giới thiệu

RAGAs (Retrieval Augmented Generation Assessment) là một framework để đánh giá các pipeline của mô hình RAG. Giữa 03 thành phần chính của mô hình RAG là Question (hay Query) - truy vấn đầu vào, Context hay (Relevant Context) - ngữ cảnh được truy xuất và Response (hay Answer) - phản hồi do LLM (Large Language Model) sinh ra cũng như Reference (hay Ground Truth Answer) có sự phụ thuộc qua các độ đo. Trong đó, nhóm thực hiện lựa chọn các độ đo có liên quan đến bài toán cần giải quyết như: Context Recall, Factual Correctness, Faithfulness và Semantic Similarity để đánh giá mô hình.



Hình 3: Độ liên quan giữa ba yếu tố trong mô hình RAG **[2]**

1. Context Recall – Độ phủ của ngữ cảnh

- Context Recall: Đo lường mức độ ngữ cảnh được truy xuất (context) bao phủ thông tin liên quan từ ngữ cảnh tham chiếu (reference).

- ***Mục đích:*** So sánh context với reference để đánh giá khả năng của hệ thống truy xuất thông tin liên quan từ ngữ cảnh.

- ***Cách đo lường:*** Thường sử dụng các độ đo như Intersection over Union (IoU) hoặc F1-Score để so sánh mức độ chồng chéo giữa ngữ cảnh được truy xuất và ngữ cảnh tham chiếu.

2. Factual Correctness – Tính chính xác về mặt dữ kiện

- **Factual Correctness:** Đánh giá tính chính xác của thông tin trong câu trả lời (response) so với ngữ cảnh được cung cấp (context).

- ***Mục đích:*** So sánh response với context để đảm bảo rằng thông tin trong câu trả lời là chính xác dựa trên ngữ cảnh được cung cấp.

- ***Cách đo lường:*** Có thể sử dụng các phương pháp như hỏi LLM đánh giá hoặc so sánh với cơ sở tri thức.

3. Faithfullness – Tính trung thực/Độ tin cậy

- **Faithfulness:** Kiểm tra xem câu trả lời (response) có được suy ra một cách trung thực từ ngữ cảnh được truy xuất (context) hay không.

- ***Mục đích:*** So sánh response với context để kiểm tra xem câu trả lời có được suy luận một cách trung thực và chỉ dựa trên ngữ cảnh được cung cấp hay không, tránh việc LLM "bịa" ra thông tin.

- ***Cách đo lường:*** Thường sử dụng các mô hình đánh giá tính nhất quán hoặc dựa vào con người đánh giá.

4. Semantic Similarity – Độ tương đồng ngữ nghĩa

- **Semantic Similarity:** So sánh ý nghĩa ngữ nghĩa giữa câu trả lời (response) và câu trả lời tham chiếu (reference).

- ***Mục đích:*** So sánh response với reference để đo lường mức độ tương đồng về ý nghĩa giữa câu trả lời được tạo ra và câu trả lời tham chiếu.

- ***Cách đo lường:*** Sử dụng các kỹ thuật như Cosine Similarity giữa các embedding của câu hoặc các metric dựa trên BERT như BERTScore.

II. Cài đặt

1. Dữ liệu đầu vào

1.1. Mô tả

- Dataset (Tập dữ liệu): Tập ngữ liệu chứa các cặp câu Hỏi-Đáp cùng ngữ cảnh mẫu sẵn có trong mô hình RAG - Vector Database.

- Queries (Tập truy vấn): Tập các câu hỏi từ người dùng truy vấn đến LLM.

1.2. Tạo Dataset và Queries

1.2.1. Tập dữ liệu gốc

- Nhóm thực hiện tạo dữ liệu đầu vào cho giai đoạn đánh giá mô hình từ tập ngữ liệu UIT-ViQuAD2.0.

- UIT-ViQuAD: Tập dữ liệu phục vụ Hỏi-Đáp tiếng Việt (Vietnamese Question Answering Dataset) với bản ban đầu gồm hơn 23,000 cặp Hỏi-Đáp về 174 chủ đề tiếng Việt từ Wikipedia. UIT-ViQuAD2.0 được bổ sung hơn 12.000 câu hỏi không trả lời được (unanswerable questions) cho cùng đoạn văn. **[3]**

1.2.2. Tạo tập dữ liệu

Nhóm thực hiện sử dụng các trường ‘context’, ‘question’ và ‘answers’ trong tập UIT-ViQuAD2.0 sau khi đã loại bỏ các unanswerable questions để tạo Dataset và Queries cho giai đoạn đánh giá. Trong đó có 4 phiên bản dữ liệu đầu vào (version):

- Version ‘in_dataset’: Tất cả queries đều có trong Dataset

+ Dataset: dataset_filtered_full – Toàn bộ tập UIT-ViQuAD2.0 sau khi đã loại bỏ các unanswerable questions

+ Queries: queries_N_uniques_in_dataset – Với mỗi context và question phân biệt, chọn ngẫu nhiên N mẫu từ dataset_filtered_full

- Version ‘out_dataset’: Tất cả queries đều <u>không</u> có trong Dataset

+ Dataset: dataset_filtered_N_dropped – Tập UIT-ViQuAD2.0 sau khi đã loại bỏ các unanswerable questions sẽ bị loại bỏ N mẫu đã được chọn ngẫu nhiên trong queries_N_uniques_in_dataset

+ Queries: queries_N_uniques_in_dataset

- Version ‘similar_dataset_within_context’: Các queries tương đồng với các queries đã có trong Dataset được sinh ra bằng question và context.

+ Dataset: dataset_filtered_full

+ Queries: queries_N_uniques_similar_dataset_within_context – Với mỗi question và context trong queries_N_uniques_in_dataset, gọi LLM (GPT mô hình gpt-4o-mini) để sinh ra N queries tương đồng.

- Version 'similar_dataset_without_context': Các queries tương đồng với các queries đã có trong Dataset được sinh ra chỉ bằng question.

+ Dataset: dataset_filtered_full

+ Queries: queries_N_uniques_similar_dataset_without_context – Chỉ với mỗi question trong queries_N_uniques_in_dataset, gọi LLM (GPT mô hình gpt-4o-mini) để sinh ra N queries tương đồng.

2. Sinh tập phản hồi

2.1. Nhúng và tạo chỉ mục Dataset

- Sau khi đọc Dataset và Queries, nhóm thực hiện tiến hành quá trình nhúng (embedding) cho các contexts và tạo chỉ mục (indexing) cho các questions trong Dataset để phục vụ cho kỹ thuật Top-k Retrieving.

- Embedding: Các contexts và questions được nhúng bởi mô hình Sentence Transformer distiluse-base-multilingual-cased-v1

- Indexing: Các embeddings của questions được tạo chỉ mục bởi mô hình Faiss Facebook AI Similarity Search (FAISS)

- Để tăng tốc độ thực thi ở các lần sau, sau khi thực hiện quá trình nhúng và tạo chỉ mục, các embeddings của contexts và questions sẽ được lưu vào tập tin PT (.pt) và index của questions được lưu vào tập tin BIN (.bin) để đọc vào các lần sau.

2.2. Sinh retrieved contexts từ Dataset và Queries

- Với các query có trong Dataset, nhóm thực hiện truy xuất trực tiếp context từ Dataset dùng làm ngữ cảnh truy xuất được (Retrieved Contexts).

- Với các query không có trong Dataset, nhóm thực hiện dùng kỹ thuật Top-k Retrieving (gọi hàm retrieve_top_k()) với k=5 để sinh các retrieved contexts.

- Top-k Retrieving: Truy xuất k contexts có liên quan nhất (Top-k Contexts) với query dựa trên một tập các contexts và một chỉ mục truy vấn (index) của Dataset:

+ Bước 1. Tìm kiếm ban đầu: Sử dụng mô hình Bi-encoder để tạo ra biểu diễn vector (embedding) cho cả query và các contexts. Bi-encoder là mô hình học được cách mã hóa văn bản thành các vector có thể so sánh được. Sau đó, sử dụng index để tìm kiếm k contexts có điểm tương đồng gần nhất với query, trả về là khoảng cách (distances) và các chỉ số (indices) của k contexts được tìm thấy.

+ Bước 2. Lọc và chuẩn bị các cặp ứng viên: Loại bỏ các contexts trùng lặp và chỉ giữ lại các cặp ứng viên duy nhất (query, context).

+ Bước 3. Xếp hạng lại các ứng viên: Sử dụng mô hình Cross-encoder để dự đoán điểm cho từng cặp ứng viên. Cross-encoder là mô hình học được cách đánh giá mức độ liên quan giữa hai đoạn văn bản. Sau đó, sắp xếp các chỉ số của các ứng viên theo thứ tự giảm dần của điểm (ranked_indices), là các ứng viên có điểm cao hơn (liên quan hơn) sẽ được xếp hạng trước.

+ Bước 4. Trả về kết quả: Duyệt qua các chỉ số đã được sắp xếp, truy xuất lại các contexts tương ứng trong danh sách ban đầu. Trả về danh sách k contexts có liên quan nhất với query, theo thứ tự giảm dần của điểm do cross-encoder đánh giá.

2.3. Sinh conversation từ retrieved contexts và query

- Với mỗi query và context, tạo conversation là đoạn đối thoại gồm các mẫu tin gồm vai trò (role) và nội dung (content) để gọi LLM sinh câu trả lời.

- Mỗi conversation gồm 02 loại mẫu tin được sử dụng:

+ Mẫu tin hệ thống: Với vai trò ‘system’, dùng để xác định nhiệm vụ của LLM.

Ví dụ: "role": "system",

"content": "You are a helpful assistant that responds in Vietnamese."

+ Mẫu tin người dùng: Với vai trò ‘user’, dùng để cung cấp question (hay query) và context cần truy vấn.

Cấu trúc: "role": "user",

"content": " Question: {query}

Context: {context}"

2.4. Sinh answer từ conversation

- Với mỗi conversation, gọi LLM (GPT) để truy vấn và nhận về answer.

- Mặc định, các tham số của mô hình GPT:

+ Mô hình: model="gpt-4o-mini",

+ Số câu trả lời nhận về: n=1 (cố định),

+ Số token tối đa: max_tokens=500,

+ Tính ngẫu nhiên: temperature=0.7

2.5. Sinh response từ answer, context và query

- Từ answer được trả lời từ LLM, context được sinh từ Dataset và query từ Queries, tạo bộ phản hồi (response) theo cấu trúc:

"user_query": query,

"retrieved_contexts": contexts,

"response": answer

- Tập phản hồi (Responses) được lưu (reponse.xlsx) để chuẩn bị cho giai đoạn Đánh giá mô hình theo RAGAs.

3. Đánh giá mô hình

- Dữ liệu đầu vào: Các trường query (từ "user_query"), contexts (từ "retrieved_contexts"), answer (từ "response") từ Responses và answer từ Queries.

- Kết hợp các dữ liệu thành bộ đầu vào cho RAGAs với cấu trúc:

"user_input": query,

"retrieved_contexts": contexts,

"response": Responses answer,

"reference": Queries answer

- Tập đầu vào được chuyển về EvaluationDataset và đánh giá qua hàm evaluate của thư viện ragas với các độ đo (metrics) **[1]**:

+ LLMContextRecall

+ Faithfulness

+ FactualCorrectness

+ SemanticSimilarity

- Kết quả trả về được lưu (evaluations.xlsx) và đánh giá trung bình (mean) trên mỗi độ đo.

III. Kết quả

Do vấn đề về thời gian thực thi và chi phí gọi LLM API (GPT mô hình mô hình gpt-4o-mini) bên cạnh gọi hàm evaluate của thư viện ragas, nhóm thực hiện chỉ chạy thử nghiệm được trên 100 mẫu cho 04 version Dataset và Queries được kết quả như sau:

1. Tổng quan

- Thời gian nhúng và tạo chỉ mục cho mỗi Dataset: ~35 phút cho ~19K mẫu gồm context và question từ Dataset.

- Thời gian Top-k Retrieving: ~50 giây cho 100 queries từ Queries không có sẵn trong Dataset.

- Thời gian gọi LLM sinh answers: ~1 phút 45 giây cho 100 conversations.

- Thời gian đánh giá RAGAs: ~4 phút cho 100 mẫu.

- Chi phí: ~0.86 USD

2. Chi tiết

2.1. Version 'in_dataset'

	Mean	**Count**
Context Recall	1	100
Factual Correctness	0.9731	100
Faithfulness	0.9143333333	100
Semantic Similarity	0.9999996653	100

2.2. Version 'out_dataset'

	Mean	**Count**
Context Recall	0.3737373737	99
Factual Correctness	0.9797979798	99
Faithfulness	0.295	100
Semantic Similarity	0.9999997231	100

2.3. Version 'similar_dataset_within_context'

	Mean	**Count**
Context Recall	0.69	100
Factual Correctness	0.952020202	99
Faithfulness	0.6033333333	100
Semantic Similarity	0.9999999628	100

2.4. Version 'similar_dataset_without_context'

	Mean	**Count**
Context Recall	0.76	100
Factual Correctness	0.9675757576	99
Faithfulness	0.6181972789	98
Semantic Similarity	0.9999962702	100

3. Nhận xét

3.1. Context Recall – Độ phủ của ngữ cảnh

$$CR_{in} = 1 > CR_{sim_without_context} > CR_{sim_within_context} > CR_{out} = 0.374$$

Context Recall của các queries sẵn có trong Dataset ('in_dataset') cao nhất và đạt tuyệt đối do các context tương ứng được truy xuất trực tiếp từ Dataset. Context Recall của các queries được sinh tương đồng với các queries trong Dataset ('similar_dataset_within_context' và 'similar_dataset_without_context') (ở mức 0.7-0.8) lớn hơn khá nhiều so với các queries không có trong Dataset ('out_dataset') (ở mức 0.3-0.4) do khi Top-k Retrieving, các contexts được truy xuất cho query không có trong Dataset thiếu chính xác và dễ bị nhiễu hơn.

3.2. Factual Correctness – Tính chính xác về mặt dữ kiện

$$FC_{out} = 0.98 > FC_{in} > FC_{sim_without_context} > FC_{sim_within_context} = 0.952$$

Factual Correctness của các queries không có và có trong Dataset cao hơn của các queries được sinh tương đồng do các response trả lời cho query tương đồng có độ chính xác thấp hơn so với khi trả lời cho query gốc. Tuy nhiên, độ lệch giữa các queries trong tất cả versions là không quá lớn, chỉ mức 0.01 giữa các versions.

3.3. Faithfullness – Tính trung thực/Độ tin cậy

$$F_{in} = 0.914 > F_{sim_without_context} > F_{sim_within_context} > F_{out} = 0.295$$

Faithfullness của các queries sẵn có trong Dataset cao nhất (ở mức 0.9) và lệch khá lớn so với các queries được sinh tương đồng (ở mức 0.6) và lệch rất lớn so với các queries không có trong Dataset (ở mức 0.3) do độ liên quan của k contexts trong Dataset được truy xuất khi Top-k Retrieving thấp đối với các queries được sinh tương đồng và rất thấp đối với các queries không có trong Dataset làm giảm sự ràng buộc giữa query và response với k contexts.

3.4. Semantic Similarity – Độ tương đồng ngữ nghĩa

$$SS_{in} \approx SS_{out} \approx SS_{sim_within_context} \approx SS_{sim_without_context} \approx 1$$

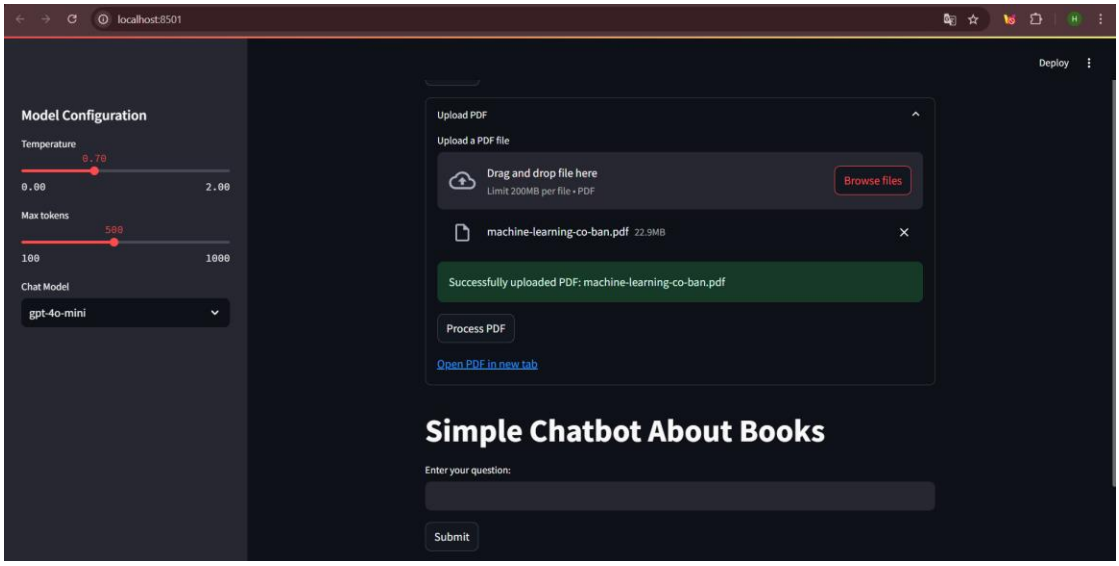
Semantic Similarity của các queries trong tất cả version xấp xỉ nhau và gần như đạt tuyệt đối do LLM thực hiện tốt việc trả lời câu hỏi, câu trả lời được sinh ra từ LLM (response) rất ít khác biệt so với câu trả lời gốc (reference) đối với cùng câu hỏi (query hay question).

***Lưu ý:** Do kết quả của version 'similar_dataset_without_context' tương đối cao hơn và không có khác biệt đáng kể so với version 'similar_dataset_within_context' nên ta có thể chỉ khảo sát trên version 'similar_dataset_without_context' để giảm thời gian và chi phí khi đính kèm context cùng question trong quá trình gọi LLM để sinh các queries tương đồng.

XÂY DỰNG ỨNG DỤNG WEB CHATBOT VỚI GIAO DIỆN NGƯỜI DÙNG

- **Mục tiêu:** Xây dựng một ứng dụng web với giao diện người dùng thân thiện, cho phép người dùng tương tác với ChatBot để đặt câu hỏi và nhận câu trả lời dựa trên nội dung của cuốn sách (file PDF).
- **Công cụ Streamlit:** Framework mã nguồn mở giúp xây dựng các ứng dụng web cho Machine Learning và Data Science một cách nhanh chóng và dễ dàng bằng Python.
- **Các thư viện và hàm đã được định nghĩa ở các bước trước:** sentence_transformers, faiss, openai, retrieve_top_k, generate_answer_chatgpt, …

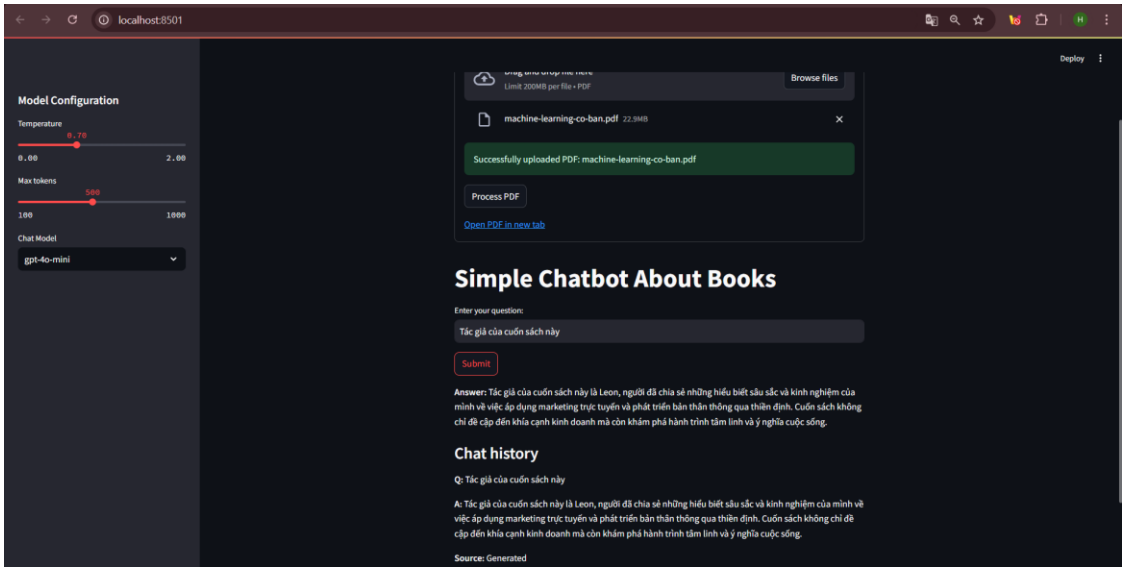
I. Thiết kế giao diện



Hình 4: Giao diện của Web ChatBot

II. Xử lý logic

- **Upload và xử lý tài liệu PDF:** Khi người dùng upload tài liệu PDF và nhấn nút "Process PDF":
- Chạy các hàm cài đặt xử lý tài liệu PDF, trích xuất thông tin, tạo ngữ cảnh và các cặp QA.
- So sánh với dữ liệu đã có sẵn trong Dataset để tìm ra các entries mới.
- Tạo embedding cho ngữ cảnh và câu hỏi mới.
- Cập nhật lại index faiss bằng các embedding mới.
- Lưu lại các file embedding đã cập nhật.
- Cập nhật dataset với dữ liệu mới.
- **Truy vấn và tạo câu trả lời:** Khi người dùng nhập câu hỏi và nhấn nút "Submit":
- Kiểm tra xem trong dataset có dữ liệu hay không.
- Khởi tạo lịch sử cho cuộc hội thoại trên hệ thống.
- Tìm k ngữ cảnh liên quan nhất đến câu hỏi.
- Kết hợp k ngữ cảnh thành một chuỗi và thêm câu hỏi, ngữ cảnh vào lịch sử cuộc hội thoại.
- Gọi hàm tạo câu trả lời dựa trên các thông tin temperature – độ "sáng tạo" của mô hình, max_tokens – số lượng tokens tối đa cho câu trả lời, chat_model – model ngôn ngữ được chọn để tạo sinh câu trả lời (gpt-4o-mini, gpt-3.5-turbo, gpt-4).



Hình 5: Giao diện upload tài liệu PDF và đặt câu hỏi

TÀI LIỆU THAM KHẢO

[1] Run ragas metrics for evaluating RAG. explodinggradients/ragas (Github):

ragas/docs/getstarted/rag_evaluation.md at main · explodinggradients/ragas · GitHub

[2] [LLM] Sử dụng RAGAs và LlamaIndex để đánh giá chất lượng RAG. Viblo AI:

[LLM] Sử dụng RAGAs và LlamaIndex để đánh giá chất lượng RAG

[3] UIT-ViQuAD2.0. taidng: taidng/UIT-ViQuAD2.0 · Datasets at Hugging Face

[4] Run ragas metrics for evaluating RAG: Evaluate Using Metrics - Ragas

[5] Ragas metrics: Metrics - Ragas